

```

`html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Excel Explorers · Kids learn spreadsheets</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: 'Comic Sans MS', 'Chalkboard SE', 'Segoe UI', cursive, sans-serif;
      background: linear-gradient(145deg, #e8f0fe 0%, #d4e3fd 100%);
      min-height: 100vh;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      padding: 20px;
      margin: 0;
    }

    .game-container {
      max-width: 1100px;
      width: 100%;
      background: rgba(255, 255, 255, 0.7);
      backdrop-filter: blur(6px);
      border-radius: 60px 60px 40px 40px;
      box-shadow: 0 20px 35px rgba(0, 20, 50, 0.2), inset 0 2px 10px rgba(255, 255, 255, 0.8);
      padding: 25px 25px 35px;
      border: 6px solid #fff9e6;
    }

    h1 {
      text-align: center;
      font-size: 3rem;
      font-weight: 700;
      color: #1e4a6b;

```

```
text-shadow: 5px 5px 0 #ffd966, 2px 2px 0 #b0c9e8;
letter-spacing: 2px;
margin-top: -5px;
margin-bottom: 15px;
display: flex;
align-items: center;
justify-content: center;
gap: 15px;
flex-wrap: wrap;
}
```

```
h1 span {
  background: #ffb347;
  border-radius: 50%;
  width: 60px;
  height: 60px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  font-size: 2.2rem;
  box-shadow: 0 8px 0 #b45f06;
  transform: rotate(8deg);
}
```

```
.subhead {
  text-align: center;
  font-size: 1.5rem;
  color: #2d6a4f;
  background: #fef9e6;
  display: inline-block;
  padding: 10px 35px;
  border-radius: 50px;
  margin: 0 auto 25px;
  border: 4px dashed #95d5b2;
  letter-spacing: 0.5px;
  box-shadow: 0 6px 0 #a3b18a;
}
```

```
.spreadsheet-area {
  background: #f8fdff;
  border-radius: 30px;
  padding: 18px;
  box-shadow: inset 0 0 0 5px #ffffff, 0 12px 0 #6c8ea0;
  border: 3px solid #3a6b8f;
```

```
margin-bottom: 25px;  
}
```

```
.excel-toolbar {  
  display: flex;  
  flex-wrap: wrap;  
  align-items: center;  
  gap: 12px;  
  background: #d9e9fa;  
  padding: 12px 20px;  
  border-radius: 40px;  
  margin-bottom: 18px;  
  border: 3px solid #ffffff;  
  box-shadow: 0 5px 0 #7c9ec0;  
}
```

```
.toolbar-btn {  
  background: #ffffff;  
  border: none;  
  padding: 10px 18px;  
  border-radius: 30px;  
  font-size: 1.2rem;  
  font-weight: bold;  
  color: #1e3c5a;  
  display: flex;  
  align-items: center;  
  gap: 8px;  
  box-shadow: 0 5px 0 #b0c7dd;  
  cursor: pointer;  
  transition: all 0.05s ease-in;  
  border: 2px solid #b6d0e8;  
  font-family: inherit;  
}
```

```
.toolbar-btn:active {  
  transform: translateY(5px);  
  box-shadow: 0 1px 0 #b0c7dd;  
}
```

```
.toolbar-btn.highlight {  
  background: #ffe68f;  
  border-color: #f5b342;  
  box-shadow: 0 5px 0 #c28f2b;  
}
```

```
.cell-ref {  
  background: white;  
  border-radius: 30px;  
  padding: 8px 22px;  
  font-size: 1.5rem;  
  font-weight: bold;  
  color: #1e4a6b;  
  border: 3px solid #aac9e4;  
  box-shadow: inset 0 2px 6px rgba(0,0,0,0.05);  
  letter-spacing: 1px;  
}
```

```
.formula-bar {  
  flex: 1;  
  min-width: 200px;  
  display: flex;  
  align-items: center;  
  background: white;  
  border-radius: 30px;  
  padding: 5px 15px;  
  border: 3px solid #aac9e4;  
  font-size: 1.4rem;  
  color: #2f4858;  
}
```

```
.formula-bar span {  
  background: #d4e3fd;  
  padding: 6px 14px;  
  border-radius: 30px;  
  margin-right: 12px;  
  font-weight: bold;  
  font-size: 1.2rem;  
}
```

```
#formulaText {  
  font-family: 'Courier New', monospace;  
  font-size: 1.3rem;  
  font-weight: 600;  
  color: #1f3a4b;  
  white-space: nowrap;  
  overflow: hidden;  
  text-overflow: ellipsis;  
}
```

```
.grid-container {  
  display: grid;  
  grid-template-columns: 60px repeat(5, 1fr);  
  gap: 4px;  
  background: #d0e2f2;  
  padding: 6px;  
  border-radius: 20px;  
  border: 4px solid #ffffff;  
  margin-bottom: 10px;  
}
```

```
.grid-header {  
  background: #bfd7f0;  
  padding: 14px 0;  
  text-align: center;  
  font-weight: bold;  
  font-size: 1.4rem;  
  color: #1e3c5a;  
  border-radius: 12px;  
  box-shadow: inset 0 -3px 0 #8ba9cc;  
}
```

```
.row-header {  
  background: #bfd7f0;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-weight: bold;  
  font-size: 1.4rem;  
  color: #1e3c5a;  
  border-radius: 12px;  
  box-shadow: inset -3px 0 0 #8ba9cc;  
}
```

```
.cell {  
  background: #ffffff;  
  border-radius: 12px;  
  min-height: 70px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-size: 1.6rem;  
  font-weight: 600;  
}
```

```
color: #1d3557;
padding: 8px;
transition: all 0.1s ease;
box-shadow: 0 3px 0 #b0c7dd;
border: 2px solid #c6d9f0;
cursor: pointer;
word-break: break-word;
text-align: center;
line-height: 1.2;
}
```

```
.cell.selected {
  outline: 6px solid #ffb347;
  outline-offset: 2px;
  transform: scale(1.02);
  box-shadow: 0 0 0 3px #ffe68f, 0 6px 0 #b0c7dd;
  z-index: 5;
  position: relative;
}
```

```
.cell.highlighted {
  background: #fff3cd;
  border-color: #f5b342;
}
```

```
.cell.active {
  transform: scale(0.98);
}
```

```
.task-area {
  background: #fffbe6;
  border-radius: 40px;
  padding: 22px 25px;
  margin: 18px 0 10px;
  border: 4px solid #ffd966;
  box-shadow: 0 10px 0 #b68b40;
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: space-between;
  gap: 20px;
}
```

```
.task-text {
```

```
font-size: 1.8rem;
font-weight: bold;
color: #5d4037;
display: flex;
align-items: center;
gap: 15px;
}
```

```
.task-text .emoji-big {
font-size: 3rem;
filter: drop-shadow(2px 4px 4px rgba(0,0,0,0.2));
}
```

```
.button-group {
display: flex;
gap: 18px;
flex-wrap: wrap;
}
```

```
.action-btn {
background: #6ec48b;
border: none;
padding: 15px 35px;
border-radius: 50px;
font-size: 1.7rem;
font-weight: bold;
color: white;
text-shadow: 2px 2px 0 #3f7a55;
box-shadow: 0 8px 0 #2e6b45;
cursor: pointer;
transition: all 0.08s;
border: 3px solid #a2e0b6;
font-family: inherit;
letter-spacing: 1px;
display: flex;
align-items: center;
gap: 10px;
}
```

```
.action-btn:active {
transform: translateY(6px);
box-shadow: 0 2px 0 #2e6b45;
}
```

```
.action-btn.secondary {  
  background: #f4a261;  
  box-shadow: 0 8px 0 #b26b2e;  
  text-shadow: 2px 2px 0 #a35f1f;  
  border-color: #fad6a5;  
}
```

```
.feedback {  
  font-size: 1.8rem;  
  font-weight: bold;  
  min-height: 3.5rem;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  gap: 12px;  
  color: #1e4a6b;  
  background: #e2f0fa;  
  padding: 8px 25px;  
  border-radius: 50px;  
  border: 4px dotted #6c8ea0;  
  margin-top: 10px;  
}
```

```
.score-badge {  
  background: #ffd966;  
  border-radius: 40px;  
  padding: 6px 25px;  
  font-size: 1.6rem;  
  font-weight: bold;  
  color: #4a2e1b;  
  display: inline-flex;  
  align-items: center;  
  gap: 10px;  
  box-shadow: inset 0 -4px 0 #c49b2b;  
}
```

</style>

</head>

<body>

<div class="game-container">

<h1>

 Excel Explorers ✨

</h1>

<div style="text-align: center;">

<div class="subhead">🐧 Click a cell · Learn formulas · Win stars!</div>

</div>

<!-- Main spreadsheet area -->

<div class="spreadsheet-area">

<!-- Toolbar with cell reference and formula bar -->

<div class="excel-toolbar">

<div class="cell-ref" id="cellRefDisplay">A1</div>

<div class="formula-bar">

fx

=

</div>

<button class="toolbar-btn" id="clearCellBtn" title="Clear selected cell">🧼 Clear</button>

</div>

<!-- Interactive grid -->

<div class="grid-container" id="excelGrid">

<!-- headers row (A-E) -->

<div class="grid-header"></div> <!-- empty corner -->

<div class="grid-header">A</div>

<div class="grid-header">B</div>

<div class="grid-header">C</div>

<div class="grid-header">D</div>

<div class="grid-header">E</div>

<!-- Row 1 -->

<div class="row-header">1</div>

<div class="cell" data-row="0" data-col="0" id="cellA1">5</div>

<div class="cell" data-row="0" data-col="1" id="cellB1">3</div>

<div class="cell" data-row="0" data-col="2" id="cellC1">=A1+B1</div>

<div class="cell" data-row="0" data-col="3" id="cellD1"></div>

<div class="cell" data-row="0" data-col="4" id="cellE1"></div>

<!-- Row 2 -->

<div class="row-header">2</div>

<div class="cell" data-row="1" data-col="0" id="cellA2">10</div>

<div class="cell" data-row="1" data-col="1" id="cellB2">7</div>

<div class="cell" data-row="1" data-col="2" id="cellC2"></div>

<div class="cell" data-row="1" data-col="3" id="cellD2"></div>

<div class="cell" data-row="1" data-col="4" id="cellE2"></div>

<!-- Row 3 -->

<div class="row-header">3</div>

<div class="cell" data-row="2" data-col="0" id="cellA3"></div>

<div class="cell" data-row="2" data-col="1" id="cellB3"></div>

```
<div class="cell" data-row="2" data-col="2" id="cellC3"></div>
<div class="cell" data-row="2" data-col="3" id="cellD3"></div>
<div class="cell" data-row="2" data-col="4" id="cellE3"></div>
</div>
```

```
<!-- Task + controls -->
<div class="task-area">
  <div class="task-text">
    <span class="emoji-big" id="taskEmoji">🎯</span>
    <span id="taskDescription">Click cell C1 — it has a formula!</span>
  </div>
  <div class="button-group">
    <button class="action-btn" id="nextTaskBtn">★ Next</button>
    <button class="action-btn secondary" id="resetBtn">↺ Reset</button>
  </div>
</div>
```

```
<!-- Feedback & score -->
<div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap;
gap: 10px;">
  <div class="feedback" id="feedbackMessage">
    <span>👏</span> Let's learn Excel!
  </div>
  <div class="score-badge">
    <span>🏆</span> <span id="starCount">0</span> / 3
  </div>
</div>
</div>
</div>
```

```
<script>
(function() {
  // ----- GAME STATE -----
  let currentTask = 0;
  let stars = 0;
  const totalTasks = 3;

  // Store data for cells (value & formula if any)
  // We'll keep the "true" values in a separate map to evaluate formulas
  // For simplicity, we only support simple formulas like =A1+B1
  const cellData = {
    'A1': { value: '5', formula: null },
    'B1': { value: '3', formula: null },
    'C1': { value: '8', formula: '=A1+B1' }, // computed
  }
})
```

```

'D1': { value: "", formula: null },
'E1': { value: "", formula: null },
'A2': { value: '10', formula: null },
'B2': { value: '7', formula: null },
'C2': { value: "", formula: null },
'D2': { value: "", formula: null },
'E2': { value: "", formula: null },
'A3': { value: "", formula: null },
'B3': { value: "", formula: null },
'C3': { value: "", formula: null },
'D3': { value: "", formula: null },
'E3': { value: "", formula: null }
};

```

```

// For reset: store original values
const initialCellData = JSON.parse(JSON.stringify(cellData));

```

```

// Currently selected cell (column letter + row number)
let selectedCellId = 'A1'; // e.g., 'A1'

```

```

// DOM elements
const cells = document.querySelectorAll('.cell');
const cellRefDisplay = document.getElementById('cellRefDisplay');
const formulaText = document.getElementById('formulaText');
const clearCellBtn = document.getElementById('clearCellBtn');
const feedbackMessage = document.getElementById('feedbackMessage');
const starCountSpan = document.getElementById('starCount');
const taskDescription = document.getElementById('taskDescription');
const taskEmoji = document.getElementById('taskEmoji');
const nextTaskBtn = document.getElementById('nextTaskBtn');
const resetBtn = document.getElementById('resetBtn');

```

```

// Helper: convert row/col to cell ID (e.g., row0 col0 -> A1)
function getCellId(row, col) {
  const colLetter = String.fromCharCode(65 + col); // A, B, C...
  const rowNum = row + 1;
  return colLetter + rowNum;
}

```

```

// Helper: get row/col from cell ID (e.g., A1 -> {row:0, col:0})
function getRowColFromId(cellId) {
  const colLetter = cellId.charAt(0);
  const rowNum = parseInt(cellId.slice(1), 10);
  return { row: rowNum - 1, col: colLetter.charCodeAt(0) - 65 };
}

```

```
}
```

```
// Evaluate simple formula like =A1+B1 (only sums, no parentheses)
```

```
function evaluateFormula(formulaStr) {
```

```
  if (!formulaStr || !formulaStr.startsWith('=')) return null;
```

```
  const expr = formulaStr.slice(1).trim(); // remove '='
```

```
  // Match simple patterns: A1+B1, A1-B1, etc. (only two operands, no spaces)
```

```
  const matches = expr.match(/^[A-Z](\d+)([+|-])([A-Z](\d+)$/i);
```

```
  if (!matches) {
```

```
    // maybe single cell reference? like =A1
```

```
    const singleMatch = expr.match(/^[A-Z](\d+)$/i);
```

```
    if (singleMatch) {
```

```
      const refId = singleMatch[1].toUpperCase() + singleMatch[2];
```

```
      const refData = cellData[refId];
```

```
      if (refData) {
```

```
        const val = refData.value;
```

```
        return isNaN(parseFloat(val)) ? val : parseFloat(val);
```

```
      }
```

```
    }
```

```
    return '#ERR'; // unsupported formula
```

```
  }
```

```
  const [, col1, row1, op, col2, row2] = matches;
```

```
  const id1 = col1.toUpperCase() + row1;
```

```
  const id2 = col2.toUpperCase() + row2;
```

```
  const val1 = cellData[id1]?.value;
```

```
  const val2 = cellData[id2]?.value;
```

```
  const num1 = parseFloat(val1);
```

```
  const num2 = parseFloat(val2);
```

```
  if (isNaN(num1) || isNaN(num2)) return '#VALUE!';
```

```
  let result;
```

```
  if (op === '+') result = num1 + num2;
```

```
  else if (op === '-') result = num1 - num2;
```

```
  else return '#OP!';
```

```
  // Return as string, but if integer, remove trailing .0
```

```
  return Number.isInteger(result) ? result.toString() : result.toFixed(2);
```

```
}
```

```
// Update all cells UI from cellData (evaluate formulas)
```

```
function refreshAllCells() {
```

```
  // First, recompute formula cells
```

```
  for (let id in cellData) {
```

```
    const data = cellData[id];
```

```
    if (data.formula) {
```

```
      const evaluated = evaluateFormula(data.formula);
```

```

        data.value = evaluated !== null ? evaluated : "";
    }
}
// Then update DOM
cells.forEach(cell => {
    const row = parseInt(cell.dataset.row, 10);
    const col = parseInt(cell.dataset.col, 10);
    const id = getCellId(row, col);
    if (cellData[id]) {
        cell.textContent = cellData[id].value;
    }
});
// Update formula bar for selected cell
updateFormulaBar();
}

// Update the formula bar and cell reference display
function updateFormulaBar() {
    if (!selectedCellId) return;
    cellRefDisplay.textContent = selectedCellId;
    const data = cellData[selectedCellId];
    if (!data) return;
    if (data.formula) {
        formulaText.textContent = data.formula;
    } else {
        formulaText.textContent = data.value !== "" ? data.value : '(empty)';
    }
}

// Highlight the selected cell in the UI
function updateSelectedCellUI() {
    cells.forEach(cell => {
        const row = parseInt(cell.dataset.row, 10);
        const col = parseInt(cell.dataset.col, 10);
        const id = getCellId(row, col);
        if (id === selectedCellId) {
            cell.classList.add('selected');
        } else {
            cell.classList.remove('selected');
        }
    });
    updateFormulaBar();
}

```

```

// Set selected cell by ID
function selectCellById(cellId) {
  if (!cellData[cellId]) return; // invalid cell
  selectedCellId = cellId;
  updateSelectedCellUI();
  // Also update task-specific highlight if needed (check later)
  checkTaskCompletion();
}

// Clear the selected cell (make empty)
function clearSelectedCell() {
  if (!selectedCellId) return;
  const data = cellData[selectedCellId];
  if (data) {
    data.value = "";
    data.formula = null;
    // Recompute any formula that depends on this cell? For simplicity, refresh all
    refreshAllCells();
    // If the cleared cell was a formula cell, we already cleared it.
    // But we may need to re-evaluate other formulas? We'll do refreshAllCells.
    // But because refreshAllCells recomputes formulas from scratch, it's fine.
  }
  // Clear any highlights from tasks
  removeTaskHighlights();
  checkTaskCompletion();
}

// Remove any 'highlighted' class from all cells (used for task hints)
function removeTaskHighlights() {
  cells.forEach(cell => cell.classList.remove('highlighted'));
}

// Apply task-specific highlight (e.g., highlight C1 in task 0)
function applyTaskHighlight() {
  removeTaskHighlights();
  if (currentTask === 0) {
    const c1 = document.getElementById('cellC1');
    if (c1) c1.classList.add('highlighted');
  } else if (currentTask === 1) {
    const a2 = document.getElementById('cellA2');
    const b2 = document.getElementById('cellB2');
    if (a2) a2.classList.add('highlighted');
    if (b2) b2.classList.add('highlighted');
  } else if (currentTask === 2) {

```

```

// Task 3: highlight C2 and D2? Actually we'll highlight the cell where they need to type
const c2 = document.getElementById('cellC2');
if (c2) c2.classList.add('highlighted');
}
}

```

// Task definitions

```

const tasks = [
  {
    description: 'Click cell C1 — it has a formula!',
    emoji: '🔍',
    check: () => {
      // Task complete if selected cell is C1
      return selectedCellId === 'C1';
    },
    hint: 'Click on cell C1 (it shows 8, but has formula =A1+B1)'
  },
  {
    description: 'Click cell A2 and B2 to see their numbers',
    emoji: '📊',
    check: () => {
      // We'll consider it complete when both A2 and B2 have been selected at least once
      // We'll use a small set to track visited cells
      if (!window.visitedCells) window.visitedCells = new Set();
      window.visitedCells.add(selectedCellId);
      return window.visitedCells.has('A2') && window.visitedCells.has('B2');
    },
    hint: 'Click on A2 (10) and B2 (7)'
  },
  {
    description: 'Select C2 and type a formula =A2+B2 (press Enter)',
    emoji

```